Reporte Técnico de Actividades Práctico-Experimentales Nro. 04

1. Datos de identificación del estudiante y la práctica

Nombre del estudiante(s)	Maria Soledad Buri Camacho
Asignatura	Simulacion
Ciclo	5
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje de la unidad	RI. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	04
Título de la Práctica	Análisis comparativo de herramientas de simulación mediante un modelo común
Nombre del Docente	José O. Guamän Q.
Fecha	22 de Octubre de 2026 23 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la práctica

Comparar diferentes lenguajes y plataformas para la implementación de modelos de simulación.

- Implementar un modelo sencillo en diferentes herramientas.

- Comparar sintaxis, facilidad de implementación y visualización.
- Identificar el ámbito de aplicación de cada herramienta

3. Materiales, reactivos, equipos y herramientas

- NumPy.
- NetLogo
- Python 3.
- Java/JDK

4. Procedimiento / Metodología Ejecutada

Para el desarrollo de la práctica se siguieron las siete etapas de un proyecto de simulación, aplicadas a la implementación del modelo epidemiológico SIR en Java y su visualización mediante NetLogo.

- Etapa 1. Definición del problema
  Se planteó la necesidad de simular la propagación de una enfermedad en una población de 100 individuos, con el objetivo de analizar la evolución de susceptibles, infectados y recuperados.
- Etapa 2. Formulación conceptual
  Se definieron los tres estados del modelo SIR (S, I y R), las transiciones entre ellos y los parámetros de contagio ($\beta = 0.30$) y recuperación ($\gamma = 0.10$), durante 100 pasos de simulación.
- Etapa 3. Recolección y análisis de datos
  Se utilizaron los parámetros establecidos en la guía: población de 100 individuos, 5 infectados iniciales y 95 susceptibles. Estos valores constituyeron las condiciones iniciales del modelo.
- Etapa 4. Construcción del modelo
  Se implementó el modelo en Java mediante la arquitectura MVC, utilizando objetos para representar individuos y números pseudoaleatorios para determinar contagios y recuperaciones. Los resultados se almacenaron en un archivo CSV.
- Etapa 5. Verificación y validación
  Se comprobó la correcta ejecución del programa, la conservación de la población total ($S + I + R = 100$) y la generación de 101 registros, correspondientes a los tiempos de 0 a 100.
- Etapa 6. Experimentación y análisis de resultados

Se ejecutó la simulación con la semilla 2026L y se importaron los resultados en NetLogo para observar gráficamente la evolución de susceptibles, infectados y recuperados.

- Etapa 7. Implementación y documentación

  Se integraron Java y NetLogo mediante el archivo resultados_java.csv. Finalmente, se organizó el proyecto, se documentaron los resultados obtenidos y se elaboraron las conclusiones y recomendaciones de la práctica.

5. Resultados

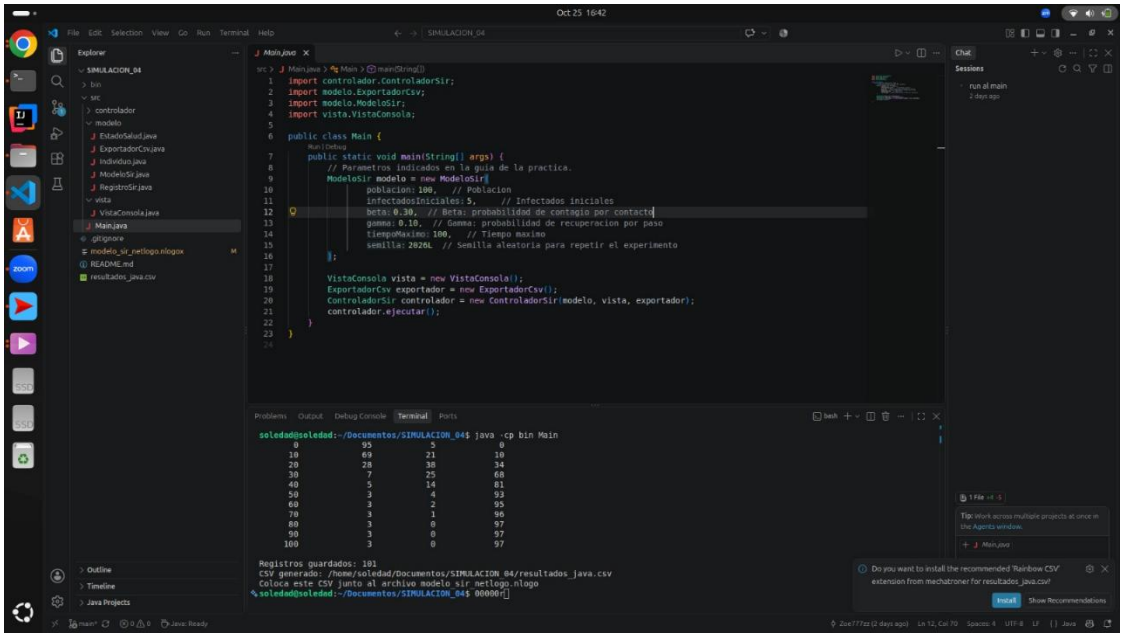


Figura 1. Estructura del código

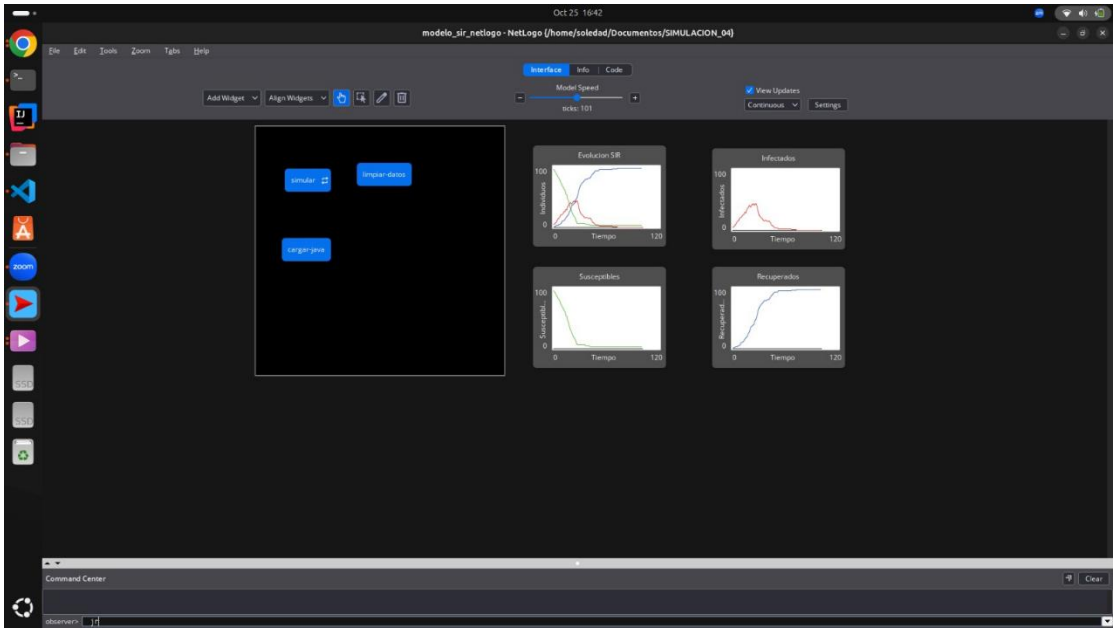


Figura 2. Implementacion de NETLOGO

Tabla a Completar

Característica	NetLogo	Java
Facilidad de implementación	Sencilla para crear botones y gráficas, aunque requeri configurar la interfaz y los procedimientos.	Moderada, debido a la organización del código mediante la arquitectura MVC y varias clases.
Manejo de agentes	En la practica se utilizó para visualizar datos del CSV.	Se representaron 100 individuos mediante objetos, cada uno con un estado de salud S, I o R.
Generación de datos	No se generó datos propios; solo importó los resultados previamente calculados por Java.	Generó los contagios y recuperaciones mediante probabilidades y registró los resultados de cada paso.
Visualización	Facilitó la representación de la evolución de susceptibles, infectados y recuperados mediante gráficas.	Mostró resultados numéricos en consola; las gráficas se realizaron posteriormente en NetLogo.
Flexibilidad	Permitió personalizar botones, colores, gráficas y la reproducción de resultados.	Permitió modificar los parámetros del modelo y organizar la lógica mediante clases independientes.
Tiempo de ejecución	La carga del CSV fue rápida y la visualización dependió de la velocidad configurada.	La simulación de 100 pasos se ejecutó rápidamente, sin dificultades apreciables.
Integración con otras herramientas	Importó el archivo resultados_java.csv para representar los resultados.	Exportó los resultados a CSV, permitiendo su integración con NetLogo.

Análisis de las ventajas y limitaciones de las herramientas

Java permitió implementar el modelo SIR mediante la arquitectura MVC, facilitando la organización del código, el cálculo de contagios y recuperaciones, y la generación de archivos CSV. Sin embargo, requirió varias clases y no incorporó gráficas. Por su parte, NetLogo facilitó la visualización de los resultados mediante gráficas y botones interactivos, aunque fue necesario configurar correctamente la importación de datos.

Análisis de las diferencias producidas por los números aleatorios

La simulación utiliza números pseudoaleatorios para determinar los contagios y las recuperaciones. En Java se empleó la semilla 2026L para obtener resultados reproducibles. Si se modifica esta semilla, los resultados pueden variar, incluso manteniendo los mismos parámetros. En nuestra práctica, NetLogo no genera nuevos resultados, sino que representa los datos de Java, por lo que ambas herramientas muestran los mismos valores de susceptibles, infectados y recuperados

6. Preguntas de Control

1. ¿Qué representa el modelo SIR?

Es un modelo epidemiológico que explica la evolución de una población distribuida entre susceptibles, infectados y recuperados.

2. ¿Qué representan los estados S, I y R?

S son personas que pueden contagiarse; I son personas infectadas actualmente; R son personas que se han recuperado y no vuelven a ser susceptibles en esta simulación.

3. ¿Qué función cumplen beta y gamma?

Beta determina la probabilidad de contagio para un contacto susceptible; gamma determina la probabilidad de recuperación de cada infectado durante un paso.

4. ¿Por qué usar los mismos parámetros en las tres herramientas?

Para comparar el comportamiento del mismo escenario, evitando que las diferencias se expliquen por poblaciones, tasas o duraciones distintas.

5. ¿Por qué Python, Java y NetLogo pueden producir resultados diferentes?

Porque los eventos dependen de números pseudoaleatorios y de detalles de programacion, como los contactos, el orden de las operaciones y la semilla. Para una comparación más justa deben igualarse esas reglas.

6. ¿Qué ventajas ofrece NetLogo en sistemas basados en agentes?

Permite representar individuos y sus interacciones en un entorno visual, observar estados y construir gráficas de resultados.

7. ¿Qué ventajas ofrece SimPy para simulaciones basadas en eventos?

Ofrece procesos, recursos y un entorno de eventos discretos que facilitan describir actividades y esperas que transcurren en el tiempo.

8. ¿Qué ventajas ofrece Java?

Tipado estático, organización por clases, reutilizacion de componentes MVC y bibliotecas estandar para datos y archivos.

9. ¿Qué función cumple el CSV en la integración?

Es un formato de intercambio para transferir los registros de Python y Java a NetLogo sin repetir la simulación.

10. ¿Qué ventajas tiene separar datos y visualización?

Permite validar la simulación de forma independiente, reutilizar el mismo CSV en diversas herramientas y modificar las gráficas sin cambiar el cálculo.

7. Conclusiones

- La implementación de la arquitectura MVC permitió organizar el código Java, separando la lógica de simulación, el control de ejecución y la presentación de resultados.
- El modelo SIR permitió representar la evolución de 100 individuos durante 100 pasos, identificando los cambios en las poblaciones de susceptibles, infectados y recuperados.
- La integración entre Java y NetLogo mediante archivos CSV facilitó la visualización de los resultados. Además, el uso de la semilla 2026L permitió obtener resultados reproducibles.

8. Recomendaciones

- Ejecutar la simulación con diferentes semillas para analizar cómo varían los contagios y las recuperaciones.
- Mantener los mismos parámetros del modelo SIR al realizar comparaciones, para identificar correctamente las diferencias entre los resultados.

9. Reflexión Critica

- La práctica permitió comprender cómo la aleatoriedad influye en la propagación de una enfermedad y cómo Java y NetLogo pueden complementarse para generar y visualizar resultados. Asimismo, se reconoció la importancia de utilizar parámetros adecuados y organizar correctamente el código para obtener una simulación comprensible y reproducible.

10. Anexos

Link del GitHub:

https://github.com/Zoe777zz/Practica_04_Simulacion_5A.git

Link del Video:

https://drive.google.com/file/d/1-43rTvTyit7pOToRFBWDyDtReAMAfyBT/view?usp=sharing

Declaración de uso de IA Generativa:

1. Herramienta de Inteligencia Artificial utilizada

- ChatGPT, desarrollada por OpenAI.

2. Forma en que se utilizó la herramienta

- Se utilizó ChatGPT como herramienta de apoyo para la implementación del modelo epidemiológico SIR en Java, mediante la arquitectura MVC. Su uso se centró la estructuración de las clases, la implementación de las probabilidades de contagio y la integración de los resultados mediante archivos CSV con NetLogo.

3. Promps usados

- Prompt 1: Ayúdame a identificar cómo dividir la estructura base de mi proyecto en Java. Recomiendame una organización de clases según sus responsabilidades, aplicando la arquitectura MVC y considerando los parámetros establecidos en la siguiente tabla
- Prompt 2: Genera un procedimiento en Java que recupere los valores de susceptibles, infectados y recuperados en cada paso de simulación y los exporte a un archivo CSV.
- Prompt 3: Guiame en NetLogo: en la importación del archivo CSV generado por Java, incorporando botones para cargar, simular y limpiar, junto con gráficas que representen la evolución de los estados S, I y R, hagamoslo paso a paso y explicándome para que sirve cada cosa.